Test Doubles

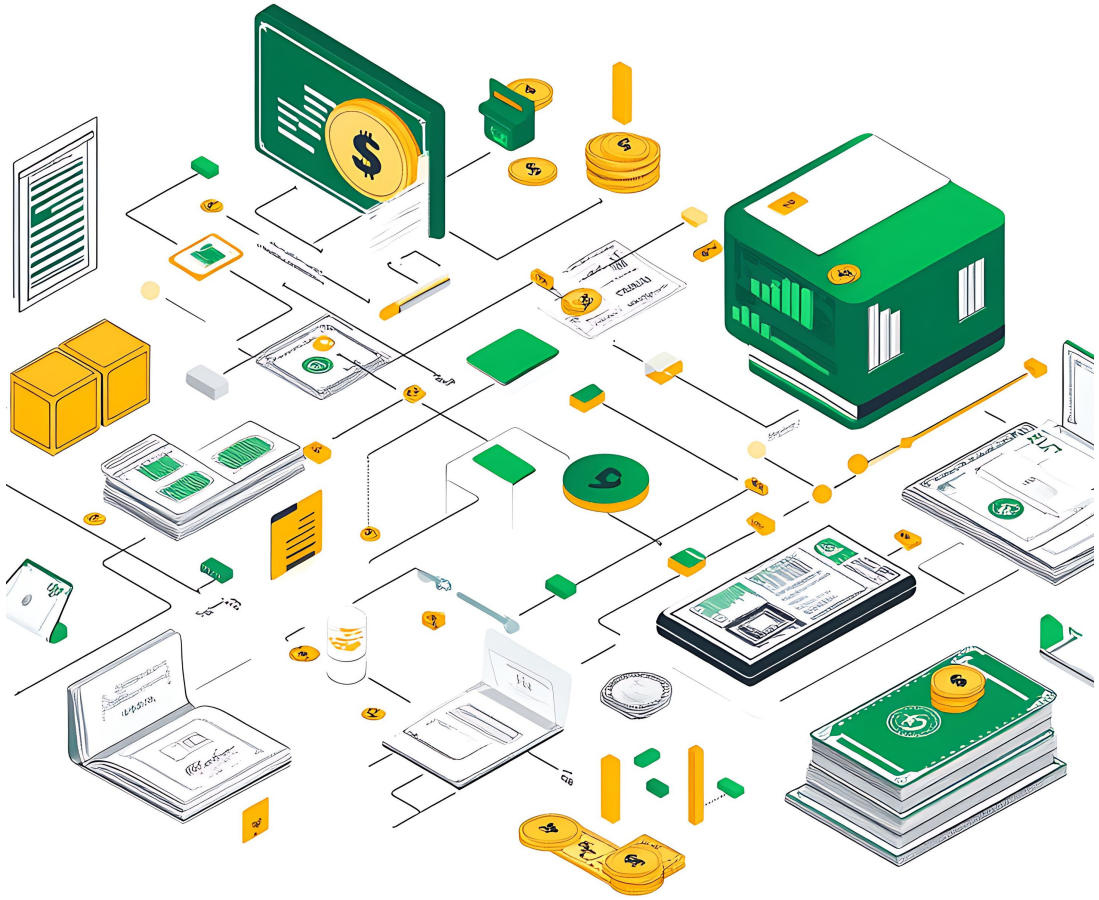
Controlling Dependencies to Produce Reliable Evidence

FinTech 512 – Software Engineering for Financial Technology

Duke

Systems are highly complex, interconnected

- Payment gateways
- Markets
- Third-party APIs
- Message Queues
- Notifications
- Blockchain networks



Why Dependencies Break Tests

Modern systems depend on things we **do not control**:

- External services and APIs
- Databases and message queues
- Time, randomness, and environmental state
- Network reliability and latency

- **Why This Is a Testing Problem**
  - Tests become **slow**
  - Results become **nondeterministic**
  - Failures become **hard to diagnose**
  - Some actions are **unsafe or irreversible**

When Real Dependencies Are the Wrong Choice

Dependency not available

- Service not implemented yet

Dependency is slow or expensive

- External APIs, databases, networks

Dependency is nondeterministic

- Time, randomness, concurrent systems

Dependency is unsafe/unrecoverable

- Sends emails, moves money, triggers workflows

Dependency behavior is irrelevant

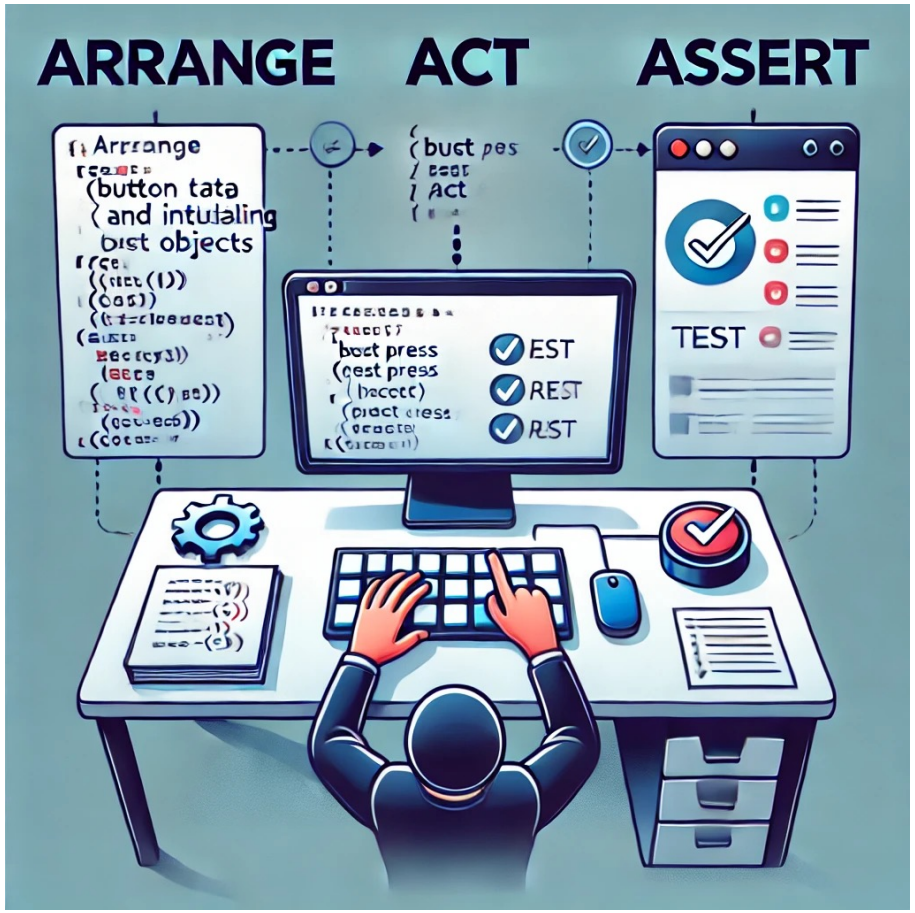
- We're testing *our* logic, not theirs

What Is a Test Double?

Controlled stand-in for a real dependency, used only during testing.

Why We Use Them

- Isolate the unit under test
- Control external behavior
- Make tests fast, safe, and repeatable



OpenAI. (2025). *ChatGPT* [Large language model]. https://chatgpt.com

Why Not Just Modify the Code for Testing?

A Tempting but Dangerous Idea

- Tests no longer exercise the **real system**
- Behavior diverges between test and production
- Bugs hide in “test-only” paths
- Confidence becomes misleading

Professional Expectation

We test the system we ship — not a special testing variant.

Seams: Where Test Doubles Enter the System

Point in code where behavior is altered without modifying the code under test.

Types:

- *Object*: Replace entire objects through inheritance or dependency injection

```
class UserService: def __init__(self, database): # database is a seam
  self.database = database
```

- *Linkage*: Mock modules/objects to link to external objects

```
@patch("requests.get")
@patch("os.getenv", return_value="fake_api_key") # Mock environment variable
def test_load_stock_quotes(mock_getenv, mock_requests_get):
    mock_response_obj = MagicMock()
    mock_response_obj.status_code = 200
    mock_response_obj.json.return_value = mock_response()
    mock_requests_get.return_value = mock_response_obj
```

Why Seams Matter

- Replace real dependencies with test doubles
- Control external behavior
- Test real production code safely

No seam → no safe way to use test doubles

Types of Test Doubles

- **Dummy**: placeholder used to satisfy method signatures
- **Stubs**: predefined response (i.e., always returns the same value)
- **Spies**: records information on how objects were called
- **Mock**: simulates behavior / verifies interaction
- **Fake / Simulator**: simplified, but working implementation

Dummy Objects

Placeholder passed to satisfy a dependency, but **never actually used**.

When to Use a Dummy

- The dependency is required by the interface
- The behavior of the dependency is irrelevant to the test
- The object exists only to allow the code to run

```
class DummyLogger:
    def log(self, message):
        pass
```

Examples

- Passing a None or empty object
- Supplying an unused logger
- Providing a required configuration object that isn't exercised

Stubs

Provides **predefined responses** to method calls.

When to Use a Stub

- The test depends on a dependency's output
- You want predictable, repeatable behavior
- You do **not** care how the dependency was called

```
class StubWeatherService:
    def get_temperature(self):
        return 72 # Always returns 72 degrees
```

Example

- A pricing service that always returns a fixed price
- A risk check that always approves or rejects
- A data lookup that returns known values

Spies

```python
class SpyEmailService:
    def __init__(self):
        self.sent_emails = []

    def send_email(self, to, subject, body):
        self.sent_emails.append({ 'to': to, 'subject': subject, 'body': body })
```

Test double that **records how it was used** so the test can inspect interactions afterward.

When to Use a Spy

- You care that a dependency was:
  - called (or not called)
  - called a specific number of times
  - called with particular arguments
- The dependency's return value may be irrelevant

Example

- Verifying a notification service was triggered
- Ensuring a trade was logged
- Confirming an audit record was written

Mocks

```python
def test_user_notification(mocker):
    mock_mailer = mocker.patch('app.mailer.send_email")
    user_service = UserService()
    user_service.notify_user("test@example.com")
    mock_mailer.assert_called_once_with("test@example.com",
                                        "Notification",
                                        "Your account has been updated")
```

Test double that **defines expectations in advance** and **fails the test if those expectations are not met**.

When to Use a Mock

- You care **how** a dependency is used
- The interaction itself is part of the contract
- Incorrect usage should immediately fail the test

Example

- Expecting a trade to be submitted exactly once
- Requiring a payment request to include specific fields
- Enforcing call order between services

Fakes/Simulators

Test double with a **working but simplified implementation**.

- Behaves like the real dependency
- Uses shortcuts or in-memory substitutes
- Safe and fast for testing

When to Use a Fake

- You want realistic behavior
- The real dependency is too slow, expensive, or unsafe
- Behavior matters more than performance fidelity

Examples

- In-memory database instead of a real DB
- Simulated payment processor
- Fake message queue
- Mock blockchain or ledger

```python
class FakeDatabase:
    def __init__(self):
        self.data = {}

    def save(self, key, value):
        self.data[key] = value

    def get(self, key):
        return self.data.get(key)
```

Test Double Anti-Patterns

Mocking what you own

- If you mock your own classes heavily, you're not testing behavior — you're testing wiring.

Over-specifying interactions

- "Called exactly 3 times in this order" → brittle tests tied to implementation.

Mocking to avoid design

- Using mocks because there's no seam (instead of fixing coupling).

Leaking doubles into production

- Test-only paths, flags, or "special modes".

Mocks for pure functions

- If there's no dependency, don't invent one.

Controlling Time

Time Is a Hidden Dependency

Many systems depend on:

- Current time
- Time zones
- Expiration windows
- Scheduling and delays

Time is:

- nondeterministic
- hard to reproduce
- a frequent source of bugs

Using a Test Double for Time

- Replace the real system clock in tests
- Freeze or control the current time
- Make time-dependent behavior predictable

Example Use Cases

- Trade cutoff times
- Order expiration
- Interest accrual
- Token expiration

Testing Dates: FreezeGun

- Simulates a specific date and time for testing
- Handles most Python APIs (datetime/arrow/pendulum)

```python
from freezegun import freeze_time
import datetime

@freeze_time("2025-01-01")
def test_function():
    assert datetime.datetime.now().year == 2025


def get_current_time():
    return datetime.datetime.now()

def test_frozen_time():
    with freeze_time("2025-06-15 10:00:00"):
        assert get_current_time() == datetime.datetime(2025, 6, 15, 10, 0, 0)
```

Controlling Databases

Databases introduce:

- Shared mutable state
- Performance variability
- Setup and teardown complexity
- Risk of data corruption

- Highly functional simulator/fake
- Store data in RAM -> avoids disk i/o overhead
- Provide real operations (CRUD) without a full-fledged server
- Consistent, fast, and isolated environment -> Docker

Using a real database in unit tests often makes tests:

- slow
- brittle
- hard to isolate

Other Types	*Database Testing Example*
Dummy	Passing None or an empty connection string to a database function.
Stub	Returning hardcoded query results without executing SQL.
Mock	Tracking if a database method (e.g., execute_query()) was called with specific parameters.
Spy	Logging queries made to the database for later verification.

```python
from sqlalchemy import create_engine, text

engine = create_engine("sqlite:///:memory:", echo=True)

with engine.connect() as conn:
    conn.execute(text("""
        CREATE TABLE company (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT,
            stock_symbol TEXT,
            ceo TEXT)"""))

    conn.execute(text("INSERT INTO company (name, stock_symbol, ceo)
                        VALUES (:name, :stock_symbol, :ceo)"),
                            [{"name": "Apple", "stock_symbol": "AAPL", "ceo": "Tim Cook"},
                             {"name": "Microsoft", "stock_symbol": "MSFT", "ceo": "Satya Nadella"}])

    conn.commit()

    result = conn.execute(text("SELECT * FROM company"))
    print(result.fetchall())
```

Controlling External APIs

External services introduce:

- Network failures
- Latency and rate limits
- Changing behavior outside your control
- Cost and compliance risk

In FinTech, APIs often:

- move money
- fetch market data
- trigger regulatory workflows

Using Test Doubles for APIs

- **Stubs**
  - Return fixed responses
- **Mocks**
  - Verify requests and parameters
- **Simulators**
  - Emulate realistic API behavior

Choosing the Right Test Double

Different Doubles Serve Different Purposes

Choosing a test double is a **design decision**, not a stylistic one.

Ask:

- What behavior do I need to **control**?
- What behavior do I need to **observe**?
- What risk am I trying to reduce?

A Practical Guide

- **Dummy:** Dependency is required but irrelevant
- **Stub:** You need controlled return values
- **Spy:** You want to observe interactions after execution
- **Mock:** Correct interaction is part of the contract
- **Fake / Simulator:** You need realistic behavior without real infrastructure

Design Lessons from Test Doubles

What Test Doubles Teach Us

- Hard-to-test code is often **tightly coupled**
- Excessive mocking can indicate:
  - unclear responsibilities
  - overly complex interactions
- Clean seams usually reflect:
  - good abstractions
  - clear contracts

Design Principles Reinforced

- **Single Responsibility Principle**
  - Easier to isolate behavior
- **Dependency Inversion Principle**
  - Dependencies depend on abstractions
- **Interfaces as Contracts**
  - Test doubles enforce expectations

If a system is hard to test, it is usually hard to change.

Slide Acknowledgements

- Dr. Brittany Johnson-Matthews, George Mason University
- Stock Photos – Adobe Education License